

Register Machine (machine)

Durante unas renovaciones de las Casamatas del Bock – una red de fortificaciones subterráneas en la ciudad de Luxemburgo – unos trabajadores han desenterrado un antiguo cofre del tesoro, sellado por un mecanismo bastante peculiar.

La inscripción de la tapa lo llama una *máquina de registros*, y explica que tiene 64 registros numerados del 0 al 63, cada uno de los cuales puede contener un entero no negativo. Inicialmente, todos los registros contienen 0.

Una colección de botones te permite introducir un *programa* que consiste en una secuencia de instrucciones. La primera instrucción que introduces está etiquetada con 0, la siguiente instrucción con 1, y así sucesivamente. Hay tres tipos de instrucción:

- **INCREMENT x , i** — El valor almacenado en el registro x aumenta en 1, y después la máquina procesa la instrucción etiquetada con i .
- **DECREMENT x , i , j** — Si el valor almacenado en el registro x es 0, la máquina procesa la instrucción etiquetada con i . En caso contrario, el registro x disminuye en 1, y después la máquina procesa la instrucción etiquetada con j .
- **HALT** — La máquina deja de procesar instrucciones.

La máquina empieza a procesar desde la instrucción etiquetada con 0. Cuando se detenga, abrirá el cofre si el valor almacenado en el registro 0 es exactamente N .

Si no se detiene después de procesar 50 000 000 instrucciones, o si intenta procesar una instrucción que no existe, la máquina sufrirá una avería catastrófica y sellará el cofre para siempre.

Tu tarea es introducir un programa que abra el cofre. Como la máquina es delicada, quieres que la longitud del programa sea lo más corta posible, y en cualquier caso no más de 100 instrucciones.

Implementación

Tendrás que enviar un único archivo fuente `.cpp`.



Entre los adjuntos de esta tarea encontrarás una plantilla `machine.cpp` y un grader de ejemplo `grader.cpp` que puedes usar para probar tus soluciones localmente.

Tendrás que implementar la siguiente función:

C++

```
void program_machine(int N)
```

- El entero N representa el número que debe quedar almacenado en el registro 0 cuando la máquina se detenga.

Tu programa puede llamar como máximo 100 veces a las siguientes funciones, que ya están definidas en el grader:

C++

```
void add_increment(int x, int i)
```

C++

```
void add_decrement(int x, int i, int j)
```

C++

`void add_halt()`

Sample Grader

Entre los adjuntos de esta tarea encontrarás una versión simplificada del grader utilizado durante la evaluación, que puedes usar para probar tus soluciones localmente. El grader de ejemplo lee datos de `stdin`, llama a la función `program_machine` y escribe en `stdout` usando el siguiente formato.

El archivo de entrada consta de una línea, que contiene el entero N .

La salida consta de una línea:

- Si tu programa abre el cofre correctamente, la salida es `OK L`, donde L es la longitud de tu programa.
- En caso contrario, la salida empieza con `WA`, seguida de una breve razón (por ejemplo, el programa excedió las 100 instrucciones, el programa no se detuvo a tiempo, etc.)

Restricciones

- $1 \leq N \leq 1000000$.

Puntuación

Tu programa será evaluado con varios casos de prueba agrupados en subtareas. Para obtener la puntuación de una subtask, tu programa debe abrir correctamente el cofre en todos los casos de prueba de esa subtask.

Para la subtask 3, la puntuación se determina por la longitud de tu programa. Sea L la máxima longitud de tu programa entre todos los casos de prueba de esta subtask. La puntuación de esta subtask es $\min(77, 100 - L)$. En particular, para obtener los 77 puntos completos de esta subtask, tu programa debe usar como máximo 23 instrucciones en todos los casos de prueba de esta subtask.

- **Subtask 0 [0 puntos]:** Casos de prueba de ejemplo.
- **Subtask 1 [7 puntos]:** $N \leq 99$.
- **Subtask 2 [16 puntos]:** N es una potencia de 2 (es decir, $N = 2^k$ para algún entero $k \geq 0$).
- **Subtask 3 [77 puntos]:** Sin restricciones adicionales. (Ten en cuenta que la puntuación de esta subtask depende de la longitud de tu programa.)

Ejemplos de entrada/salida

Grader

Solución

`program_machine(3)`

```
add_increment(1, 1)
add_increment(0, 2)
add_decrement(1, 4, 1)
add_halt()
add_increment(0, 3)
```

Grader

Solución

```
program_machine(5)
```

```
add_increment(0, 1)
add_increment(0, 2)
add_increment(0, 3)
add_increment(0, 4)
add_increment(0, 5)
add_halt()
```

Explicación

En el primer ejemplo, $N = 3$. El ejemplo crea un programa con 5 instrucciones:

0.	INCREMENT 1, 1
1.	INCREMENT 0, 2
2.	DECREMENT 1, 4, 1
3.	HALT
4.	INCREMENT 0, 3

La siguiente tabla muestra la etiqueta de cada instrucción que procesa la máquina y el contenido de los registros 0 y 1 después:

Instrucción	Registro 0	Registro 1
—	0	0
0	0	1
1	1	1
2	1	0
1	2	0
2	2	0
4	3	0
3	3	0

En el segundo ejemplo, $N = 5$ y el programa de ejemplo mostrado usa 6 instrucciones.